
boolean



Middleware

Come mettere i bastoni tra le ruote...

Middleware

Express fornisce una serie di strumenti per le applicazioni web, come le **richieste** e le **risposte** HTTP, il **routing** e i **middleware**. Grazie a questi strumenti possiamo creare virtualmente qualsiasi applicazione.

Cosa sono i middleware?

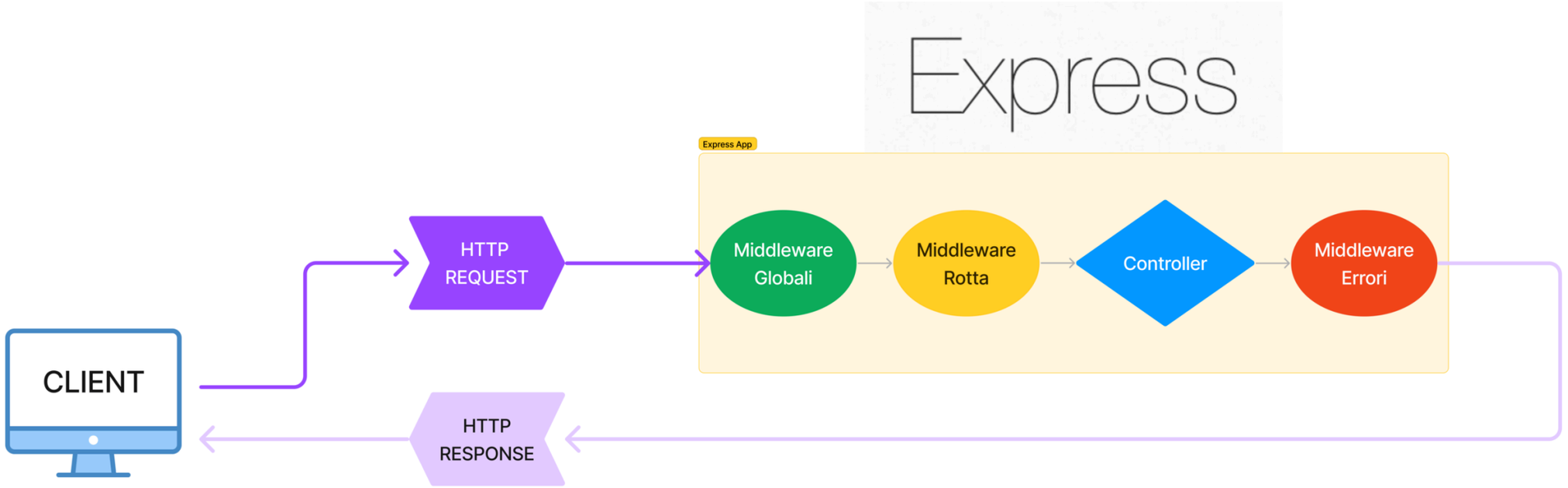
Sono **funzioni** che vengono eseguite prima che una richiesta HTTP raggiunga il gestore del percorso o prima che un client riceva una risposta, dando al framework la possibilità di eseguire uno script prima o dopo la richiesta del client.

Grazie a questi possiamo intercettare il flusso dell'applicazione. Possiamo ad esempio verificare se un utente è loggato o se ha i permessi per eseguire una determinata azione o accedere ad una pagina.

Middleware

Quando vengono eseguiti i middleware?

A seconda del tipo, questi possono essere eseguiti prima dell'azione associata ad una rotta o dopo.



Middleware

Si ma come si usano?

L'utilizzo dei middleware, si svolge in **2 steps:**

1) Dichiarazione della funzione middleware

Qui definiamo la sua logica

2) Registrazione (o assegnazione) del middleware

Questo step serve per far sapere ad Express che un middleware esiste e per quali rotte deve essere utilizzato

Middleware - Dichiarazione

Come si creano?

I middleware sono delle semplici funzioni (nominate o anonime) che ricevono 3 argomenti:

- **req** e **res** rapprensentano rispettivamente la Richiesta e Risposta HTTP
- **next** rappresenta la prossima funzione che dovrà essere eseguita nel flusso della richiesta

```
1  function checkTime(req, res, next) {
2    // se l'ora è dopo le 22:00 e prima delle 7:00
3    // rispondiamo con un messaggio di errore
4    const currHour = new Date().getHours();
5
6    if (currHour >= 22 || currHour < 7) {
7      return res.send("Meglio se vai a dormire!");
8    }
9
10   next();
11 }
12
13 module.exports = checkTime
```

Middleware - Dichiarazione

E se dimentico la funzione next?

La funzione **next()** si occupa di far proseguire il flusso della richiesta e se non viene invocata, la richiesta rimane in stallo.

Qualora volessimo bloccare una richiesta intenzionalmente è importante invocare il metodo **res.send()** o simili, per inviare una **response** che informa il client del motivo per cui l'abbiamo bloccata.

```
1  function checkTime(req, res, next) {
2    // se l'ora è dopo le 22:00 e prima delle 7:00
3    // rispondiamo con un messaggio di errore
4    const currHour = new Date().getHours();
5
6    if (currHour >= 22 || currHour < 7) {
7      return res.send("Meglio se vai a dormire!");
8    }
9
10   next();
11 }
12
13 module.exports = checkTime
```

Middleware - Registrazione

Registrazione globale

Per poter usare un middleware, è necessario registrarlo!

I middleware possono essere registrati a livello **globale** tramite la funzione **use()**.

Così facendo, il middleware verrà chiamato **per ogni rotta della nostra applicazione**

*In **App.js** (globale)*

```
1 const express = require('express')
2 const app = express()
3 const checkTime = require('./middlewares/checkTime')
4
5 // registrato per tutte le rotte dell'app
6 app.use(checkTime)
```

Middleware - Registrazione

Registrazione per path

Oltre a specificare i middleware per singola rotta possiamo anche specificarli **in base al percorso**, indipendentemente dalla rotta.

Alla funzione **use()** come primo argomento passiamo il **path** che vogliamo monitorare.

La rotta e tutte le rotte figlie di quel path utilizzeranno il middleware indicato *indipendentemente dal metodo HTTP utilizzato.*

```
1 const express = require('express')
2 const app = express()
3
4 //registrato per tutte le rotte che iniziano con /pizzas
5 app.use("/pizzas", checkTime);
```

Middleware - Registrazione

Registrazione per singola rotta

Possono anche essere registrati a livello di **singola rotta** specificando il middleware come secondo argomento della funzione che la definisce.

In questo caso il middleware viene eseguito **esclusivamente per quella specifica rotta!**

```
1 const express = require('express')
2 const app = express()
3 const checkTime = require('./middlewares/checkTime')
4
5 // registrato solo per questa rotta
6 app.get("/", checkTime, (req, res) => {
7   res.send("Hello World!");
8 });
```

Middleware - Registrazione

Registrazione per router

I middleware possono essere anche essere registrati a livello di **router** tramite la funzione **use()**.

Ad esempio, assegnandolo al router delle pizze, verrebbe eseguito **per ogni rotta che riguarda le pizze**, **ma non per le altre!**

```
1 const express = require('express')
2 const app = express()
3 const checkTime = require('../middlewares/checkTime')
4 const router = express.Router()
5
6
7 // registrato per le rotte di questo router
8 router.use(checkTime)
```

LIVE CODING

Proviamo a registrare un middleware in vari modi

Middleware - Applicazioni pratiche

Tutto molto bello ma... a che servono?

Abbiamo detto che i **middleware** sono funzioni che **vengono eseguite prima che la richiesta arrivi a destinazione.**

Questo vuol dire che possiamo fare delle operazioni **preliminari** affinchè la richiesta arrivi nel modo corretto... ci viene in mente niente?

Middleware - Applicazioni pratiche

Tutto molto bello ma... a che servono?

Abbiamo detto che i **middleware** sono funzioni che **vengono eseguite prima che la richiesta arrivi a destinazione.**

Questo vuol dire che possiamo fare delle operazioni **preliminari** affinchè la richiesta arrivi nel modo corretto... ci viene in mente niente?

Ad esempio, **potremmo controllare eventuali errori...**

Middleware - Applicazioni pratiche

Tutto molto bello ma... a che servono?

Abbiamo detto che i **middleware** sono funzioni che **vengono eseguite prima che la richiesta arrivi a destinazione.**

Questo vuol dire che possiamo fare delle operazioni **preliminari** affinchè la richiesta arrivi nel modo corretto... ci viene in mente niente?

Ad esempio, **potremmo controllare eventuali errori...**

e ancora, **potremmo convertire i dati** della richiesta in modo che arrivino col giusto formato...

Middleware - Applicazioni pratiche

Ma allora.....

Vuol dire che i **body parser** e gli **asset statici** sono in realtà dei middleware?

```
1 const express = require('express')
2 const app = express()
3
4 // registriamo gli assets statici
5 app.use(express.static("public"));
6
7 // application/json
8 app.use(express.json());
```

Middleware - Applicazioni pratiche

Ma allora.....

Vuol dire che i **body parser** e gli **asset statici** sono in realtà dei middleware?



```
const express = require('express')
const app = express()

// registriamo gli assets statici
app.use(express.static("public"));

// application/json
app.use(express.json());
```

Middleware - applicazioni pratiche

Dulcis in fundo

Proviamo a usare i middleware per *intercettare i possibili errori* dell'applicazione.

Creeremo una funzione come quella degli altri middleware, ma questa avrà **4 argomenti**, dove il primo sarà **err** mentre gli altri saranno *req, res* e *next*.

***Nota:** non usiamo **next()** perchè non vogliamo che la richiesta prosegua ma che risponda con 500*

Infine registriamo il middleware **globalmente** e **in fondo a tutte le altre rotte**

```
1 function errorsHandler(err, req, res, next) {
2   res.status(500)
3   res.json({
4     error: err.message,
5   });
6 };
7
8 module.exports = errorsHandler;
```

```
1 const express = require("express");
2 const app = express();
3 const errorsHandler = require("./middlewares/errorsHandler");
4
5 // Tutte le altre rotte...
6
7
8 // Poi
9 app.use(errorsHandler);
```

Middleware - applicazioni pratiche

E se viene chiamata una rotta inesistente?

Creiamo un middleware che risponda adeguatamente, con un messaggio d'errore e uno status **404 - Not Found**

Notiamo come in questo caso:

- non usiamo **err** perchè per Express non è un errore chiamare una rotta inesistente
- non usiamo **next()** perchè non vogliamo che la richiesta prosegua ma che risponda con 404

Successivamente registriamo il middleware **globalmente** e **in fondo a tutte le altre rotte, anche dopo quella di gestione errori**

```
function notFound (req, res, next) {
  res.status(404)
  res.json({
    error: "Not Found",
    message: "Pagina non trovata"
  });
};

module.exports = notFound;
```

```
const express = require("express");
const app = express();
const notFound = require("./middlewares/notFound");

// Tutte le altre rotte...


// Poi
app.use(notFound);
```

LIVE CODING

Creazione ed utilizzo dei middleware con gestione degli errori



ESERCITAZIONE